

FRAUD360

Deployment Instructions

Building and releasing Fraud360 on private-cloud virtual machines

Version 1.1 · 2 October 2026

Finverge · Confidential — for the customer's infrastructure and security teams

Document control

Item	Detail
Document	Fraud360 — Deployment Instructions (private cloud, VM-based)
Version	1.1 — 2 October 2026 (supersedes 1.0)
Companion	Fraud360 — Infrastructure Requirements v1.1 (what to provision; sizes, network, security, DR)
Platform baseline	Fraud360 as in BRD v1.9, FSD v1.6, HLD v1.8
Audience	Finverge delivery engineers and the customer's Linux, database and network administrators
Changes in 1.1	Roles are data. Two data migrations now seed role rows and permissions (§7.2, new §7.6 check); every authorising service must reach tenant-service for role lookups (§8.2); first-tenant, acceptance-test, upgrade and troubleshooting sections extended (§14.1, §15.4, §17.1, §18).
Status	Runbook for first deployment into UAT, then production. Written from the code base and standard tooling; not yet executed end to end on a customer platform — see §1.3

Contents

1. Purpose and conventions	5
1.1 Conventions	5
1.2 Variables and example addressing	5
1.3 What has and has not been verified	5
2. Deployment overview	6
3. Phase 0 — Pre-flight checks	7
4. Phase 1 — Base OS build (all VMs)	8
4.1 Packages and service account	8
4.2 Directory layout	8
4.3 Kernel limits and firewall	8
4.4 Monitoring agent	9
4.5 Exit check	9
5. Phase 2 — Database cluster	10
5.1 Install (db-01, db-02)	10
5.2 etcd (three members)	10
5.3 Patroni (db-01, db-02)	10
5.4 Virtual IP	11
5.5 Backups (pgBackRest)	12
5.6 Exit check	12
6. Phase 3 — Build and package the release	13
6.1 Assemble and install	13
6.2 Verify and package	13
7. Phase 4 — Database objects	14

7.1 Owner role and database	14
7.2 Run migrations as the owner.....	14
7.3 Create the service roles	14
7.4 Replace the shared temporary password with one per role	15
7.5 Exit check — isolation is real	15
7.6 Exit check — every tenant has its roles	15
8. Phase 5 — Secrets and configuration.....	17
8.1 Generate the platform secrets	17
8.2 Common environment file.....	17
8.3 Per-service environment files	18
8.4 Worker environment files.....	19
8.5 Exit check	19
9. Phase 6 — Install the release and start the app VMs.....	20
9.1 Install the release (every app, dec and wrk VM)	20
9.2 The service unit.....	20
9.3 Start order on app VMs	20
9.4 Exit check (on each app VM)	20
10. Phase 7 — Decision VMs (Lane A)	22
11. Phase 8 — Workers and scheduled jobs.....	23
11.1 Continuous workers.....	23
11.2 Scheduled jobs (wrk-01)	23
11.3 Exit check	24
12. Phase 9 — Load balancers	26
12.1 HAProxy (lb-01 and lb-02, identical)	26
12.2 Internal VIP listeners.....	26
12.3 ISO 8583 pass-through (only if in scope)	27
12.4 keepalived.....	27
12.5 Exit check	28
13. Phases 10 and 11 — SFTP and monitoring	29
13.1 SFTP landing zone (sftp-01)	29
13.2 Monitoring (mon-01)	29
14. Phase 12 — First tenant and functional verification	31
14.1 Administrator and tenant	31
14.2 End-to-end checks	31
15. Phase 13 — Acceptance tests	32
15.1 Load and latency (BR-309 / BR-909)	32
15.2 Failover and resilience	32

15.3 Backup, restore and DR (BR-908) 32

15.4 Security checks 33

16. Phase 14 — Go-live..... 34

17. Day-two operations 35

17.1 Routine release (upgrade) 35

17.2 Rollback..... 35

17.3 Rotating secrets 35

17.4 Scaling out 36

17.5 Disaster recovery promotion 36

17.6 Failing the timers over to wrk-02 36

18. Troubleshooting..... 37

Appendix A — Environment variable reference 38

Appendix B — Port and unit map 38

1. Purpose and conventions

This is the build and release runbook. It takes the infrastructure described in the Infrastructure Requirements document from “VMs exist” to “Fraud360 is live, verified and operable”, in a fixed order, with an exit check at the end of every phase. Do not skip an exit check; later phases assume it passed.

1.1 Conventions

- Commands are for RHEL 9 and its rebuilds. Ubuntu differences are noted where they matter (`apt` for `dnf`, `ufw` or `nftables` for `firewalld`, `postgresql-16` package names).
- Commands prefixed `#` run as root (or with `sudo`); `$` runs as the unprivileged `fraud360` service user. Run privileged steps from the bastion with a named account.
- Angle-bracket tokens such as `<BANK>` are values to substitute. Shell variables used throughout are in §1.2.
- Secrets are never typed into a command line that lands in shell history. Generate them into files with restrictive permissions, or read them from the vault.
- Every step that changes state is safe to re-run unless it says otherwise.

1.2 Variables and example addressing

Replace with the customer’s real names and addresses from the Phase 0 checklist.

Variable	Example	Meaning
<code>FX_VER</code>	<code>2026.10.0</code>	Release identifier; the directory name under <code>/opt/fraud360/releases</code>
<code>BANK_DOMAIN</code>	<code>bank.internal</code>	Internal DNS suffix
<code>CONSOLE_HOST</code>	<code>fraud360.bank.internal</code>	Console and API name → external VIP
<code>DECIDE_HOST</code>	<code>decide.fraud360.bank.internal</code>	Lane A name → same external VIP, routed by host
<code>DB_HOST</code>	<code>db.fraud360.bank.internal</code>	Database VIP (follows the Patroni leader)
<code>INT_VIP</code>	<code>10.20.0.5</code>	Internal service VIP on the load balancers
<code>EXT_VIP</code>	<code>10.10.0.5</code>	External service VIP
<code>app-01 / app-02</code>	<code>10.20.1.11 / .12</code>	App VMs
<code>dec-01 / dec-02</code>	<code>10.20.2.11 / .12</code>	Decision VMs
<code>wrk-01 / wrk-02</code>	<code>10.20.3.11 / .12</code>	Worker VMs
<code>db-01 / 02 / 03</code>	<code>10.30.0.11 / .12 / .13</code>	Database nodes and witness

1.3 What has and has not been verified

Treat the first run as a rehearsal

The service start commands, environment variables, database roles, health endpoints, job scripts and their arguments in this document were taken from the repository (`run_local.ps1`, `settings.py`, the compose overlay, the scripts’ own documentation). The operating-system, Patroni, HAProxy, keepalived and systemd configuration is standard practice written for this layout, and **has not been executed on the customer’s platform**. Run the whole of this document against UAT first. Phase 12 exists to find what differs.

2. Deployment overview

Phase	What happens	VMs	Exit check
0	Pre-flight: network, DNS, certificates, time, storage, access	All	Every check in §3 passes
1	Base OS build: users, directories, kernel settings, firewall, NFS, monitoring agent	All	Hardening and mount checks pass
2	Database cluster: PostgreSQL 16, Patroni, VIP, backups	db-01..03	Switchover works; first backup succeeds
3	Build and package the release	Build host	Checksum recorded; imports succeed
4	Database objects: owner role, migrations, service roles, unique passwords	db VIP	Role-isolation test passes
5	Secrets and configuration files	app, dec, wrk	No default secret remains
6	Install release; app VMs; start services	app-01/02	Local <code>/health/ready</code> is 200
7	Decision VMs	dec-01/02	<code>/health/ready</code> is 200
8	Workers and scheduled jobs	wrk-01/02	Backlog drains; each job runs under its role
9	Load balancers and VIPs	lb-01/02	Both VIPs serve; failover works
10	SFTP landing zone	sftp-01	A test drop is ingested
11	Monitoring and alerting	mon-01	Every target up; test alert fires
12	First tenant and functional verification	—	Console, ingestion, alert, case, Lane C path all work
13	Acceptance: load, failover, restore, security	—	All acceptance criteria in §15 pass
14	Go-live and hand-over	—	Sign-off

Order matters in three places: the database must exist before any service starts (the tenant service seeds its first administrator at startup); migrations run before the new code starts, never after; and the load balancer is configured last so nothing is published until the pools are healthy.

3. Phase 0 — Pre-flight checks

Run these from the bastion and from one VM in each zone. A failed check is a prerequisite problem for the customer's team, not something to work around in the application.

Check	Command	Expected
Time sync	<code>chronyc tracking</code>	Reference set; offset under 100 ms
DNS	<code>dig +short fraud360.<BANK>.internal db.fraud360.<BANK>.internal</code>	Resolve to the intended VIPs
Reverse path / zone rules	<code>nc -zv <db-01> 5432</code> from a client network; then from app-01	Refused from client network; open from app-01 once the DB is up
Certificate	<code>openssl verify -CAfile ca.pem fullchain.pem; openssl x509 -noout -dates -subject -ext subjectAltName -in cert.pem</code>	OK; SANs cover <code>CONSOLE_HOST</code> and <code>DECIDE_HOST</code> ; at least 90 days left
Disk encryption	<code>lsblk -f</code> and <code>cryptsetup status <mapper></code> on db, sftp and NFS hosts	Data and WAL volumes encrypted
Storage	<code>showmount -e <nas>;</code> test mount; <code>df -h</code>	Export visible; 1 TB free
Egress	<code>curl -sS -o /dev/null -w '%{http_code}' --proxy <proxy> https://<feed-url></code> per allow-listed feed	200 / expected status, or a documented decision not to enable the feed
Vault	Retrieve and discard one test secret from each required path	Succeeds with the deployment identity
Python on build host	<code>python3.12 --version; pip download --no-deps torch==2.14.0+cpu --extra-index-url <mirror> -d /tmp/x</code>	3.12.x; wheel resolves from the CPU index
Tesseract on wrk VMs	<code>tesseract --version; tesseract --list-langs</code>	Version prints; English present

4. Phase 1 — Base OS build (all VMs)

4.1 Packages and service account

Apply the bank's hardened image first. Then, on app, dec and wrk VMs:

```
# dnf -y install python3.12 python3.12-pip libxml2 libxslt nfs-utils chrony firewalld policycoreutils-  
python-utils  
# dnf -y install tesseract tesseract-langpack-eng # wrk VMs (OCR for Lane C)  
# systemctl enable --now chronyd firewalld  
  
# Fixed numeric IDs so file ownership is identical on every VM and over NFS  
# groupadd -g 3600 fraud360  
# useradd -u 3600 -g fraud360 -M -s /sbin/nologin -d /opt/fraud360 fraud360
```

4.2 Directory layout

Path	Purpose	Owner / mode
/opt/fraud360/releases/<FX_VER>/	One immutable directory per release (code + virtualenv)	root:fraud360 755
/opt/fraud360/current	Symlink to the active release	root
/etc/fraud360/	Environment files holding secrets	root:root 700; files 600
/srv/fraud360/shared/	NFS mount: lane_c_statements/, drop/, spool/	root:fraud360 755 (export root and drop/ must be root-owned for SFTP chroot); lane_c_statements/, spool/ fraud360:fraud360 2770
/var/lib/fraud360/var/	Local state: notifications/, models/	fraud360:fraud360 750
/var/log/fraud360/	Optional file logs (journald is primary)	fraud360:fraud360 750

```
# install -d -m 755 -o root -g fraud360 /opt/fraud360/releases  
# install -d -m 700 /etc/fraud360  
# install -d -m 750 -o fraud360 -g fraud360 /var/lib/fraud360/var/notifications  
/var/lib/fraud360/var/models /var/log/fraud360  
  
# echo '<nas>:/fraud360 /srv/fraud360/shared nfs4 rw,hard,noatime,sec=sys,_netdev 0 0' >> /etc/fstab  
# mount /srv/fraud360/shared  
# Do these once, as root, on a host that mounts the export without root squash (or on the NAS itself):  
# install -d -m 755 -o root -g fraud360 /srv/fraud360/shared/drop  
# install -d -m 2770 -o fraud360 -g fraud360 /srv/fraud360/shared/lane_c_statements  
/srv/fraud360/shared/spool
```

4.3 Kernel limits and firewall

```
# cat > /etc/sysctl.d/90-fraud360.conf <<'EOF'  
net.core.somaxconn = 4096  
net.ipv4.tcp_max_syn_backlog = 4096  
net.ipv4.ip_local_port_range = 10240 65000  
vm.swappiness = 10  
EOF  
# sysctl --system  
  
# On app VMs: allow service ports only from the LB, decision, worker and monitoring subnets  
# firewall-cmd --permanent --new-zone=fx
```

```
# firewall-cmd --permanent --zone=fx --add-source=<lb-subnet> --add-source=<dec-subnet> --add-source=<wrk-  
subnet> --add-source=<mon-subnet>  
# firewall-cmd --permanent --zone=fx --add-port=8080-8088/tcp  
# firewall-cmd --permanent --zone=fx --add-port=9100/tcp  
# firewall-cmd --reload
```

Equivalent rules apply to dec VMs (8086, 8588, 9100) and wrk VMs (9100 only; workers have no inbound ports). SELinux stays **enforcing**; if a unit is denied, fix the label or add a targeted policy module rather than disabling it.

4.4 Monitoring agent

Install `node_exporter` from the bank's approved repository on every VM and enable it (port 9100, source-restricted to mon-01).

4.5 Exit check

```
$ id fraud360                                # uid=3600 gid=3600  
# mount | grep fraud360/shared                # nfs4 ... rw  
# sudo -u fraud360 touch /srv/fraud360/shared/spool/.probe && echo writable && rm  
/srv/fraud360/shared/spool/.probe  
# getenforce                                  # Enforcing  
# chronyc tracking | grep -E 'Leap|System time'
```

5. Phase 2 — Database cluster

Fraud360 needs only this from PostgreSQL: version 14 or later (16 recommended), one database, the owner role created in Phase 4, encrypted storage, and a highly-available endpoint. If the customer's DBA team already operates a PostgreSQL platform with its own standard (Patroni, repmgr, EDB or an equivalent), use that, satisfy the requirements in Infrastructure Requirements §5.5, and go to Phase 3. The reference build below uses Patroni with etcd.

5.1 Install (db-01, db-02)

```
# dnf -y install https://download.postgresql.org/pub/repos/yum/reporpms/EL-9-x86_64/pgdg-redhat-repo-latest.noarch.rpm
# dnf -qy module disable postgresql
# dnf -y install postgresql16-server postgresql16-contrib patroni patroni-etcd pgbackrest
# systemctl disable postgresql-16                # Patroni owns the lifecycle

# On db-01, db-02, db-03:
# dnf -y install etcd && systemctl enable etcd
```

Patroni and etcd packages differ by repository; use the versions and sources the bank has approved. Create the data directory on the encrypted data volume and the WAL directory on the separate WAL volume:

```
# install -d -m 700 -o postgres -g postgres /pgdata/16/data /pgwal/16
```

5.2 etcd (three members)

Configure `/etc/etcd/etcd.conf` on each of db-01, db-02, db-03 with its own name and the three peer URLs, then start it. Use TLS between members if the bank requires it.

```
ETCD_NAME=db-01
ETCD_DATA_DIR=/var/lib/etcd
ETCD_LISTEN_PEER_URLS=http://10.30.0.11:2380
ETCD_LISTEN_CLIENT_URLS=http://10.30.0.11:2379,http://127.0.0.1:2379
ETCD_ADVERTISE_CLIENT_URLS=http://10.30.0.11:2379
ETCD_INITIAL_ADVERTISE_PEER_URLS=http://10.30.0.11:2380
ETCD_INITIAL_CLUSTER=db-01=http://10.30.0.11:2380,db-02=http://10.30.0.12:2380,db-03=http://10.30.0.13:2380
ETCD_INITIAL_CLUSTER_STATE=new
```

```
# systemctl start etcd
# etcdctl --endpoints=10.30.0.11:2379,10.30.0.12:2379,10.30.0.13:2379 endpoint health
```

5.3 Patroni (db-01, db-02)

```
# /etc/patroni/patroni.yml (db-01; change name and addresses on db-02)
scope: fraud360
name: db-01
restapi: { listen: 0.0.0.0:8008, connect_address: 10.30.0.11:8008 }
etcd3: { hosts: '10.30.0.11:2379,10.30.0.12:2379,10.30.0.13:2379' }
bootstrap:
  initdb: [ { encoding: UTF8 }, data-checksums, { waldir: /pgwal/16 } ]
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576
    synchronous_mode: true
    postgresql:
      use_pg_rewind: true
```

```

parameters:
  max_connections: 500
  shared_buffers: 16GB
  effective_cache_size: 44GB
  work_mem: 32MB
  maintenance_work_mem: 2GB
  wal_level: replica
  max_wal_size: 16GB
  checkpoint_timeout: 15min
  random_page_cost: 1.1
  archive_mode: 'on'
  archive_command: 'pgbackrest --stanza=fraud360 archive-push %p'
  ssl: 'on'
  ssl_cert_file: /etc/pki/fraud360/db.crt
  ssl_key_file: /etc/pki/fraud360/db.key
postgresql:
  listen: 0.0.0.0:5432
  connect_address: 10.30.0.11:5432
  data_dir: /pgdata/16/data
  bin_dir: /usr/pgsql-16/bin
  authentication:
    replication: { username: replicator, password: '<from vault>' }
    superuser: { username: postgres, password: '<from vault>' }
  pg_hba:
    - local all postgres peer
    - hostssl replication replicator 10.30.0.0/24 scram-sha-256
    - hostssl controlplane all 10.20.0.0/16 scram-sha-256 # application zone only
    - host all all 0.0.0.0/0 reject

```

`archive_command` fails until the pgBackRest stanza exists (§5.5), and WAL accumulates meanwhile; create the stanza immediately after the first start. The `waldir` `initdb` option places WAL on the separate volume for the primary; confirm in UAT that the replica's base backup lands WAL on its own WAL volume too, or size the data volume to hold both.

Start Patroni on db-01 first, wait for it to become leader, then on db-02:

```

# systemctl enable --now patroni
# patronictl -c /etc/patroni/patroni.yml list
# expect: db-01 Leader running, db-02 Sync Standby streaming, lag 0

```

synchronous_mode and availability

With `synchronous_mode: true` Patroni keeps a synchronous standby so a failover loses no committed data. If the standby is lost, Patroni demotes to asynchronous rather than blocking writes unless `synchronous_mode_strict` is set. Leave strict mode off unless the bank prefers to halt rather than risk a gap.

5.4 Virtual IP

Install `vip-manager` on db-01 and db-02 so that `DB_HOST` follows the Patroni leader:

```

# /etc/default/vip-manager
VIP_IP=10.30.0.10
VIP_NETMASK=24
VIP_INTERFACE=ens192
VIP_TRIGGER_KEY=/service/fraud360/leader
VIP_DCS_TYPE=etcd
VIP_DCS_ENDPOINTS=http://10.30.0.11:2379,http://10.30.0.12:2379,http://10.30.0.13:2379

# systemctl enable --now vip-manager
$ ping -c1 10.30.0.10 # DB_HOST resolves to this address

```

5.5 Backups (pgBackRest)

```
# /etc/pgbackrest.conf
[global]
repo1-path=/backup/pgbackrest
repo1-retention-full=2
repo1-retention-diff=14
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<from vault>
process-max=4
[fraud360]
pg1-path=/pgdata/16/data

# once, on the leader:
$ sudo -u postgres pgbackrest --stanza=fraud360 stanza-create
$ sudo -u postgres pgbackrest --stanza=fraud360 check
$ sudo -u postgres pgbackrest --stanza=fraud360 --type=full backup

# schedule (cron or systemd timer on the leader's host):
# Sun 01:00 full | Mon-Sat 01:00 diff | WAL archived continuously
```

Point **repo1-path** at the bank's backup repository (a mounted share in a separate failure domain, or an S3-compatible endpoint using **repo1-type=s3** inside India). Replicate the repository to the DR site. The cipher passphrase lives in the vault: a backup that cannot be decrypted is not a backup.

5.6 Exit check

```
$ patronictl -c /etc/patroni/patroni.yml switchover --candidate db-02 --force # then switch back
$ psql 'host=10.30.0.10 user=postgres dbname=postgres sslmode=require' -c 'select pg_is_in_recovery()'
$ sudo -u postgres pgbackrest --stanza=fraud360 info
```

Pass when: switchover completes and the VIP follows; the old leader rejoins as a standby; one full backup is listed.

6. Phase 3 — Build and package the release

Build once, on a build VM with the same OS and Python as the targets and with the install path used in production. Python virtual environments embed absolute paths, so a virtualenv built at `/opt/fraud360/releases/<FX_VER>` must be installed at that same path. The same artefact is then deployed to every VM and every environment; only the environment files differ.

6.1 Assemble and install

```
$ export FX_VER=2026.10.0
$ install -d /opt/fraud360/releases/$FX_VER && cd /opt/fraud360/releases/$FX_VER
$ tar xzf /staging/fraud360-src-$FX_VER.tar.gz          # services/ packages/ alembic/ alembic.ini
scripts/ requirements.txt

$ python3.12 -m venv .venv
$ .venv/bin/pip install --upgrade pip
$ .venv/bin/pip install \
    --index-url https://<internal-mirror>/simple \
    --extra-index-url https://<internal-mirror>/pytorch-cpu \
    -r requirements.txt
$ .venv/bin/pip install -e packages/cp_common
```

Package contents. The source archive contains `services/`, `packages/`, `alembic/`, `alembic.ini`, `scripts/` and `requirements.txt`. Exclude `tests/`, `docs/`, `frontend/`, `node_modules/`, `data/`, `drop/`, `spool/`, `var/`, log files, sample PDFs and any `.env`. The shipped console is under `services/gateway/app/static/` and needs no build step.

CPU-only PyTorch

`requirements.txt` selects the CPU build (`torch==2.14.0+cpu`) through an extra index. If the mirror does not carry it, pip may resolve a CUDA build that is many times larger. Check the installed version (`.venv/bin/python -c "import torch;print(torch.__version__, torch.cuda.is_available())"`) — it should print a `+cpu` version and `False`.

6.2 Verify and package

```
$ .venv/bin/python -c "import cp_common, fastapi, sqlalchemy, psycpg2, torch, pdfplumber, pytesseract;
print('imports ok')"
$ PYTHONPATH=. .venv/bin/python -c "import services.gateway.app.main, services.decision_service.app.main,
\
    services.lane_c_service.app.main, services.analytics_service.app.main; print('apps import')"

$ cd /opt/fraud360/releases
$ tar czf /staging/fraud360-$FX_VER-linux.tar.gz $FX_VER
$ sha256sum /staging/fraud360-$FX_VER-linux.tar.gz | tee /staging/fraud360-$FX_VER-linux.sha256
```

Record the checksum in the change record. Every VM verifies it before extracting (§9.1). Do not run the application test suite against the production database; it belongs on the build host against a throwaway database.

7. Phase 4 — Database objects

Run from a worker VM or the bastion with the release extracted and the PostgreSQL client installed. These steps are the only ones that use a superuser or the owner credential interactively.

7.1 Owner role and database

```
# As postgres on the leader. Choose and vault the owner password first.
$ psql 'host=<DB_HOST> user=postgres dbname=postgres sslmode=require' <<'SQL'
CREATE ROLE cp LOGIN PASSWORD '<OWNER_PASSWORD>';
CREATE DATABASE controlplane OWNER cp;
SQL
```

7.2 Run migrations as the owner

Migrations create the ten schemas and every table. They run as the owner role, never as a service role: a process that can alter tables while serving traffic can also drop them.

```
$ cd /opt/fraud360/releases/$FX_VER
$ export PYTHONPATH=$PWD
$ export
DATABASE_URL='postgresql+psycopg2://cp:<OWNER_PASSWORD_URL_ENCODED>@<DB_HOST>:5432/controlplane?sslmode=require'
$ .venv/bin/python -m alembic current
$ .venv/bin/python -m alembic upgrade head
$ .venv/bin/python -m alembic current          # should print the head revision
$ .venv/bin/python -m alembic check           # models match the schema (expects no diff)
```

A URL-encoded password is required if it contains characters such as @, :, / or %.

Two of these migrations are data migrations

`e2b5c8d1f4a7` adds `permissions` and `description` to `tenant.tenant_roles` and makes sure every existing tenant has a row for each of the ten starter roles, with the permissions the old code granted. `f3c6d9e2a5b8` adds the sanctions-screening and borrower-credit-health permissions to them. Together they are what lets the platform authorise anyone at all: roles are read only from those rows, and there is no list of roles in code to fall back on. They are safe to re-run, they never overwrite a permission a tenant has edited, and `scripts/backfill_tenant_roles.py` is no longer needed. Take the pre-migration backup (§17.1) before running them on a database with real tenants.

7.3 Create the service roles

`provision_db_roles.py` creates one login role per service, grants it only its own schemas (plus `platform`), sets a role-level `search_path`, and sets default privileges so tables created by later migrations are reachable without re-running it. It runs **after** the first migration because it grants on schemas the migrations create.

```
$ .venv/bin/python scripts/provision_db_roles.py \
  --superuser-url
  'postgresql+psycopg2://postgres:<PG_SUPER>@<DB_HOST>:5432/controlplane?sslmode=require' \
  --password "$(openssl rand -base64 24)"          # temporary; replaced per role in §7.4
```

Role	Reaches only
svc_tenant	tenant, platform
svc_branding	branding, platform

Role	Reaches only
svc_config	config, platform
svc_analytics	analytics, cases, ingestion, platform (the one deliberate cross-service grant: it reads the ingestion queue)
svc_ingestion	ingestion, platform
svc_notify	notify, platform
svc_decision	decision, platform
svc_laneec	lane_c, platform

7.4 Replace the shared temporary password with one per role

The provisioning script applies a single password to all eight roles. That is acceptable for a few minutes and not for operation. Generate a distinct secret per role, store it in the vault, and apply it:

```
$ for r in svc_tenant svc_branding svc_config svc_analytics svc_ingestion svc_notify svc_decision
svc_laneec; do
    pw=$(openssl rand -base64 24 | tr -d '/+= ' | cut -c1-28)          # URL-safe, no encoding needed
    psql 'host=<DB_HOST> user=postgres dbname=controlplane sslmode=require' -qc "ALTER ROLE $r PASSWORD
'$pw'"
    vault kv put secret/fraud360/db/$r password="$pw"                # or the bank's equivalent
done
```

ALTER ROLE ... PASSWORD takes effect for new connections only; nothing is running yet, so there is no overlap to manage. In later rotations, change the role, update the env file, then restart (§17.3).

7.5 Exit check — isolation is real

```
# Each must FAIL with 'permission denied' or 'relation does not exist':
$ psql "host=<DB_HOST> user=svc_decision dbname=controlplane sslmode=require" -c 'select count(*) from
tenant.tenants'
$ psql "host=<DB_HOST> user=svc_laneec    dbname=controlplane sslmode=require" -c 'select count(*) from
analytics.fact_case'
# Each must SUCCEED:
$ psql "host=<DB_HOST> user=svc_tenant    dbname=controlplane sslmode=require" -c 'select count(*) from
tenant.tenants'
# The default password from the repository must be rejected:
$ PGPASSWORD=cp_password psql "host=<DB_HOST> user=svc_tenant dbname=controlplane sslmode=require" -c
'select 1'
```

7.6 Exit check — every tenant has its roles

```
$ psql 'host=<DB_HOST> user=postgres dbname=controlplane sslmode=require' <<'SQL'
-- every tenant should have at least the ten starter roles (more if it has added its own)
SELECT t.slug, count(r.id) AS roles
FROM tenant.tenants t LEFT JOIN tenant.tenant_roles r ON r.tenant_id = t.id
GROUP BY t.slug ORDER BY roles, t.slug;

-- and the starter roles must carry their grants (should list ten roles; only data_scientist is expected
to show 0 actions)
SELECT name, json_array_length(permissions) AS actions FROM tenant.tenant_roles
WHERE tenant_id = (SELECT id FROM tenant.tenants LIMIT 1) ORDER BY name;
SQL
```

A tenant with no rows would have nobody who can do anything. If one appears, re-run **alembic upgrade head**; a tenant onboarded after this point gets its roles from the onboarding flow itself.

8. Phase 5 — Secrets and configuration

8.1 Generate the platform secrets

```
# JWT signing secret and internal key: 48 random bytes each
$ openssl rand -base64 48          # JWT_SECRET
$ openssl rand -base64 48          # INTERNAL_API_KEY

# Application-layer encryption key (AES-256-GCM): version:base64(32 bytes)
$ python3 -c "import os,base64; print('1:'+base64.b64encode(os.urandom(32)).decode())"      #
CP_ENCRYPTION_KEYS
```

Store all three in the vault. `JWT_SECRET` and `INTERNAL_API_KEY` must be **identical on every service and worker**. `CP_ENCRYPTION_KEYS` must be set before the first tenant user enrolls MFA; it guards MFA seeds, IdP client secrets and feed credentials. Keep it outside the database and out of backups that sit beside the database — a stolen backup must not carry its own key. Do **not** set `CP_ALLOW_PLAINTEXT_SECRETS`.

8.2 Common environment file

`/etc/fraud360/common.env`, root-owned, mode 600. systemd reads it as root, so the service account never needs read access to secrets on disk.

```
PLATFORM_NAME=<Bank> Fraud360
CONSOLE_BASE_URL=https://fraud360.<BANK>.internal
JWT_SECRET=<vault>
JWT_EXPIRE_MINUTES=30
INTERNAL_API_KEY=<vault>
CP_ENCRYPTION_KEYS=1:<vault>

CACHE_BACKEND=database
APP_REPLICAS=2
ANALYTICS_ENGINE=postgres
AUTO_CREATE_TABLES=false
SEED_DEMO_TENANT=false
SEED_ADMIN_EMAIL=<named platform administrator>
SEED_ADMIN_PASSWORD=<one-time, from vault>
SEED_ADMIN_MUST_CHANGE_PASSWORD=true

LANE_C_STORAGE_DIR=/srv/fraud360/shared/lane_c_statements
INGEST_DROP_ROOT=/srv/fraud360/shared/drop
REPORT_SPOOL_DIR=/srv/fraud360/shared/spool
GEOLOCATE_URL=<internal geolocation service, or leave unreachable>
PYTHONUNBUFFERED=1
```

`SEED_ADMIN_PASSWORD` seeds the first platform administrator at tenant-service startup and the account is forced to change it at first sign-in. After the administrator has signed in, remove `SEED_ADMIN_PASSWORD` from the file.

Service URLs differ by host type. On **app** VMs each service talks to its neighbours on the same VM; on **dec** and **wrk** VMs they use the internal VIP:

Variable	On app VMs	On dec and wrk VMs
<code>TENANT_SERVICE_URL</code>	<code>http://127.0.0.1:8081</code>	<code>http://<INT_VIP>:8081</code>
<code>BRANDING_SERVICE_URL</code>	<code>http://127.0.0.1:8082</code>	<code>http://<INT_VIP>:8082</code>
<code>CONFIG_SERVICE_URL</code>	<code>http://127.0.0.1:8083</code>	<code>http://<INT_VIP>:8083</code>
<code>ANALYTICS_SERVICE_URL</code>	<code>http://127.0.0.1:8084</code>	<code>http://<INT_VIP>:8084</code>
<code>INGESTION_SERVICE_URL</code>	<code>http://127.0.0.1:8085</code>	<code>http://<INT_VIP>:8085</code>
<code>DECISION_SERVICE_URL</code>	<code>http://<INT_VIP>:8086</code>	<code>http://127.0.0.1:8086</code> on dec; <code>http://<INT_VIP>:8086</code> on wrk

Variable	On app VMs	On dec and wrk VMs
NOTIFICATION_SERVICE_URL	http://127.0.0.1:8087	http://<INT_VIP>:8087
LANE_C_SERVICE_URL	http://127.0.0.1:8088	http://<INT_VIP>:8088

Every authorising service needs tenant-service

A service decides what a caller may do from that tenant's role rows, which it fetches over HTTP from tenant-service (the only service allowed to read that schema) and caches against a shared generation. That makes `TENANT_SERVICE_URL` and `INTERNAL_API_KEY` mandatory on the analytics, config, Lane C, notification and decision services and on every worker that calls them, not just the gateway. On dec and wrk VMs the URL is the internal VIP (`http://<INT_VIP>:8081`), so the internal listener for 8081 (§12.2) must exist. If tenant-service cannot be reached and a service has never cached the tenant's roles, it refuses the request rather than guessing: after a deployment, a wave of 403 `role_not_permitted` almost always means this URL or key is wrong.

Use literal IP addresses, not `localhost`: on a dual-stack host `localhost` can resolve to `::1` first, and the gateway's two-second connect budget makes that intermittent and hard to diagnose.

8.3 Per-service environment files

One file per service, root:root 600, in `/etc/fraud360/<unit>.env`:

```
# /etc/fraud360/tenant-service.env
APP_MODULE=services.tenant_service.app.main:app
PORT=8081
DATABASE_URL=postgresql+psycopg2://svc_tenant:<vault>@<DB_HOST>:5432/controlplane?sslmode=require
DB_SEARCH_PATH=tenant,platform
```

File	APP_MODULE	PORT	DB role	DB_SEARCH_PATH
gateway.env	services.gateway.app.main:app	8080	(none; no DATABASE_URL)	—
tenant-service.env	services.tenant_service.app.main:app	8081	svc_tenant	tenant,platform
branding-service.env	services.branding_service.app.main:app	8082	svc_branding	branding,platform
config-service.env	services.config_service.app.main:app	8083	svc_config	config,platform
analytics-service.env	services.analytics_service.app.main:app	8084	svc_analytics	analytics,cases,ingestion,platform
ingestion-service.env	services.ingestion_service.app.main:app	8085	svc_ingestion	ingestion,platform
decision-service.env	services.decision_service.app.main:app	8086	svc_decision	decision,platform
notification-service.env	services.notification_service.app.main:app	8087	svc_notify	notify,platform
lane-c-service.env	services.lane_c_service.app.main:app	8088	svc_lanec	lane_c,platform
iso8583-gateway.env	services.iso8583_gateway.app.main:app	8589 (HTTP health); TCP listener 8588 via <code>ISO8583_GATEWAY_PORT</code>	(none)	—

The ISO 8583 gateway additionally needs `ISO8583_CLIENT_ID`, `ISO8583_CLIENT_SECRET`, `ISO8583_TENANT_ID`, `ISO8583_TZ_OFFSET_MINUTES` (330 for IST) and an explicit `ISO8583_FAIL_OPEN=true|false`. There is deliberately no default for the last one: whether a switch that gets no answer treats it as approve or as issuer-inoperative is the bank's decision, recorded before go-live. The service refuses to start without it.

The ISO 8583 gateway is one process with two listeners: the TCP listener for switches (8588 by default) and an HTTP port for `/health` and `/metrics`, which must differ from it, so `PORT=8589` in its env file. Add 8589 to the monitoring scrape list.

8.4 Worker environment files

Workers use the same pattern with their own role. Create `worker-analytics.env`, `worker-lanec.env`, `worker-ingestion.env` and `worker-owner.env`:

File	DB role and search path	Used by
worker-analytics.env	svc_analytics — analytics,cases,ingestion,platform	detection, reference fetch, board pack, OFAC load
worker-lanec.env	svc_lanec — lane_c,platform	Lane C worker, Lane C reference fetch
worker-ingestion.env	svc_ingestion — ingestion,platform	file intake
worker-owner.env	cp — owner; search path analytics,cases,tenant,branding,config,platform,public	notification watcher, subscriptions, usage roll-up, retention, ring scores, seal_secrets

`worker-owner.env` is the most sensitive file on the estate other than the vault. It exists only on worker VMs and the bastion.

8.5 Exit check

```
# grep -RIIE 'change-me|ChangeMe|cp_password|dev-secret|dev-internal-key' /etc/fraud360 # must print
nothing
# stat -c '%a %U:%G %n' /etc/fraud360 /etc/fraud360/* # 700 / 600
root:root
```

9. Phase 6 — Install the release and start the app VMs

9.1 Install the release (every app, dec and wrk VM)

```
# cd /staging && sha256sum -c fraud360-$FX_VER-linux.sha256
# tar xzf fraud360-$FX_VER-linux.tar.gz -C /opt/fraud360/releases
# chown -R root:fraud360 /opt/fraud360/releases/$FX_VER && chmod -R g-w,o-rwx
/opt/fraud360/releases/$FX_VER
# ln -sf /opt/fraud360/releases/$FX_VER /opt/fraud360/current
# ln -sf /var/lib/fraud360/var /opt/fraud360/releases/$FX_VER/var      # relative 'var/' paths resolve
to local state
```

9.2 The service unit

One template unit starts any service. The instance name selects `/etc/fraud360/<instance>.env`.

```
# /etc/systemd/system/fraud360@.service
[Unit]
Description=Fraud360 %i
After=network-online.target remote-fs.target
Wants=network-online.target

[Service]
User=fraud360
Group=fraud360
WorkingDirectory=/opt/fraud360/current
Environment=PYTHONPATH=/opt/fraud360/current
EnvironmentFile=/etc/fraud360/common.env
EnvironmentFile=/etc/fraud360/%i.env
ExecStart=/opt/fraud360/current/.venv/bin/python -m uvicorn ${APP_MODULE} --host 0.0.0.0 --port ${PORT}
Restart=always
RestartSec=3
LimitNOFILE=65536
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true
ReadWritePaths=/srv/fraud360 /var/lib/fraud360 /var/log/fraud360

[Install]
WantedBy=multi-user.target
```

`--reload` is a development flag used by `run_local.ps1`; never use it here.

9.3 Start order on app VMs

Start **tenant-service first**: at startup it seeds the first platform administrator and requires the database and its role. Then the rest. The gateway goes last because its readiness probes its upstreams.

```
# systemctl daemon-reload
# systemctl enable --now fraud360@tenant-service
# sleep 5; journalctl -u fraud360@tenant-service -n 30 --no-pager      # look for seed completion, no
tracebacks
# for s in config-service branding-service ingestion-service notification-service analytics-service lane-
c-service; do
    systemctl enable --now fraud360@$s; done
# systemctl enable --now fraud360@gateway
```

9.4 Exit check (on each app VM)

```
$ for p in 8081 8082 8083 8084 8085 8087 8088 8080; do
    printf '%s ' $p; curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:$p/health/ready; done
```

```
# all 200
$ curl -s http://127.0.0.1:8080/health
$ curl -s -H "X-Internal-Key: <INTERNAL_API_KEY from the vault>" http://127.0.0.1:8084/metrics | head
$ curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8084/metrics # 403 without the key
$ curl -s http://127.0.0.1:8080/ | head -c 200 # the console's
index.html
```

If a service reports not ready, `journalctl -u fraud360@<name>` shows why. The common causes are a wrong password or search path in its env file, an unreachable database VIP, or `APP_REPLICAS` above 1 with `CACHE_BACKEND=memory`, which the service refuses at startup.

10. Phase 7 — Decision VMs (Lane A)

```
# (install the release as in §9.1)
# systemctl enable --now fraud360@decision-service
# systemctl enable --now fraud360@iso8583-gateway          # only if ISO 8583 is in scope and the fail-
open decision is recorded
$ curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8086/health/ready
```

The decision service owns its counters and decision log in its own schema and answers within the rail budget; it does not call the analytics store. `JWT_SECRET` must match the tenant service's, because the machine tokens that authorise `/decide` are issued by the tenant service and verified here.

Lane A needs a written failure policy

Each inline rail has a `fail` policy with no default (and `ISO8583_FAIL_OPEN` for ISO 8583). Until the bank has decided what happens when no decision can be returned, do not point a production switch at this service.

11. Phase 8 — Workers and scheduled jobs

11.1 Continuous workers

```
# /etc/systemd/system/fraud360-detect@.service      # instances 1..N; N from Infrastructure Requirements
$5.3
[Unit]
Description=Fraud360 detection worker %i
After=network-online.target
[Service]
User=fraud360
Group=fraud360
WorkingDirectory=/opt/fraud360/current
Environment=PYTHONPATH=/opt/fraud360/current
EnvironmentFile=/etc/fraud360/common.env
EnvironmentFile=/etc/fraud360/worker-analytics.env
ExecStart=/opt/fraud360/current/.venv/bin/python scripts/run_detection.py --loop 30
Restart=always
RestartSec=3
NoNewPrivileges=true
PrivateTmp=true
[Install]
WantedBy=multi-user.target
```

The other continuous workers use the same unit body with a different **EnvironmentFile** and **ExecStart**:

Unit	ExecStart arguments (after python)	EnvironmentFile	Hosts
fraud360-detect@1..N	scripts/run_detection.py --loop 30	worker-analytics.env	wrk-01, wrk-02
fraud360-lanec	scripts/run_lane_c.py --loop 300	worker-lanec.env	wrk-01, wrk-02
fraud360-notify	scripts/run_notifications.py --loop 300	worker-owner.env	wrk-01 only

```
# systemctl daemon-reload
# systemctl enable --now fraud360-detect@1 fraud360-detect@2 fraud360-lanec
# systemctl enable --now fraud360-notify                # wrk-01 only
```

Detection workers claim work with **FOR UPDATE SKIP LOCKED**, so adding an instance on either VM simply takes a further slice of the queue. Where a worker calls ingestion's internal claim/acknowledge API it does so over **INGESTION_SERVICE_URL** (the internal VIP), authenticated with **INTERNAL_API_KEY**.

11.2 Scheduled jobs (wrk-01)

Each job is a oneshot service plus a timer. The timers are enabled on **wrk-01 only**; wrk-02 holds the same unit files, disabled, as a warm standby (§17.6). The table is the specification; the generator below implements it.

Job name	ExecStart arguments	Env file	OnCalendar (host time)
ofac	scripts/load_ofac_sdn.py	worker-analytics	*-*-* 01:30:00
usage-rollup	scripts/run_usage_rollup.py	worker-owner	*-*-* 00:20:00
reference-fetch	scripts/run_reference_fetch.py --commit	worker-analytics	*-*-* 02:00:00
lanec-reference-fetch	scripts/run_lane_c_reference_fetch.py --commit	worker-lanec	*-*-* 02:30:00
board-pack	scripts/run_board_pack.py --commit	worker-analytics	*-*-* 03:00:00
ring-scores	scripts/refresh_ring_scores.py	worker-owner	Sun *-*-* 04:00:00
retention-preview	scripts/run_retention.py (dry run; no --commit)	worker-owner	Sun *-*-* 05:00:00
subscriptions	scripts/run_subscriptions.py --once	worker-owner	*-*-* *:05:00 (hourly)

Job name	ExecStart arguments	Env file	OnCalendar (host time)
file-intake	<code>scripts/run_file_intake.py --once</code>	worker-ingestion	<code>*-*-* *:*:00</code> (every minute)

A small generator writes one explicit service-and-timer pair per job, so each job names its own environment file:

```
# /root/make-fx-jobs.sh (run once; edit the table to add jobs)
while IFS='|' read -r name args envfile when; do
cat > /etc/systemd/system/fraud360-job-$name.service <<EOF
[Unit]
Description=Fraud360 job $name
[Service]
Type=oneshot
User=fraud360
Group=fraud360
WorkingDirectory=/opt/fraud360/current
Environment=PYTHONPATH=/opt/fraud360/current
EnvironmentFile=/etc/fraud360/common.env
EnvironmentFile=/etc/fraud360/$envfile.env
ExecStart=/opt/fraud360/current/.venv/bin/python $args
TimeoutStartSec=3h
NoNewPrivileges=true
PrivateTmp=true
EOF
cat > /etc/systemd/system/fraud360-job-$name.timer <<EOF
[Unit]
Description=Fraud360 job $name schedule
[Timer]
OnCalendar=$when
Persistent=true
RandomizedDelaySec=30
[Install]
WantedBy=timers.target
EOF
done <<'TABLE'
ofac|scripts/load_ofac_sdn.py|worker-analytics|*-*-* 01:30:00
usage-rollup|scripts/run_usage_rollup.py|worker-owner|*-*-* 00:20:00
reference-fetch|scripts/run_reference_fetch.py --commit|worker-analytics|*-*-* 02:00:00
lanec-reference-fetch|scripts/run_lane_c_reference_fetch.py --commit|worker-lanec|*-*-* 02:30:00
board-pack|scripts/run_board_pack.py --commit|worker-analytics|*-*-* 03:00:00
ring-scores|scripts/refresh_ring_scores.py|worker-owner|Sun *-*-* 04:00:00
retention-preview|scripts/run_retention.py|worker-owner|Sun *-*-* 05:00:00
subscriptions|scripts/run_subscriptions.py --once|worker-owner|*-*-* *:05:00
file-intake|scripts/run_file_intake.py --once|worker-ingestion|*-*-* *:00:00
TABLE

# systemctl daemon-reload
# systemctl enable --now $(ls /etc/systemd/system/fraud360-job-*.timer | xargs -n1 basename)
```

Retention is deliberately two-step

The scheduled job is a **dry run** that writes what it would delete to the journal. A compliance officer reviews that output, and only then is `scripts/run_retention.py --commit` (optionally `--classes <list>`) run, as an approved change, as the owner role. A preview that does not match what executes is worse than none, so the same predicates serve both.

11.3 Exit check

```
# systemctl list-timers 'fraud360-job-*' --no-pager
# for j in ofac reference-fetch board-pack subscriptions usage-rollup retention-preview file-intake; do
#     systemctl start fraud360-job-$j.service; systemctl is-failed fraud360-job-$j.service; done
# journalctl -u 'fraud360-job-*' --since '10 min ago' --no-pager | grep -iE
# 'denied|permission|error|traceback'
```

This is where a wrong role assignment surfaces. A `permission denied for schema ...` means that job needs a different env file; move it and re-run. Pass when every job exits cleanly, and the detection queue age (Phase 12) is near zero.

12. Phase 9 — Load balancers

12.1 HAProxy (lb-01 and lb-02, identical)

Install HAProxy 2.4 or later and build the certificate bundle (`cat fullchain.pem privkey.pem > /etc/haproxy/certs/fraud360.pem`, mode 640 root:haproxy). HAProxy gives the active `/health/ready` checks the design depends on, so an app VM whose local service has failed is drained, not merely retried.

```
# /etc/haproxy/haproxy.cfg
global
    log /dev/log local0
    maxconn 20000
    ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
    mode http
    log global
    option httplog
    option forwardfor
    option redispatch
    retries 2
    timeout connect 3s
    timeout client 30s
    timeout server 30s

frontend fe_https
    bind 10.10.0.5:443 ssl crt /etc/haproxy/certs/fraud360.pem alpn h2,http/1.1
    unique-id-format %{+X}o%ci:%cp_%fi:%fp_%Ts_%rt:%pid
    http-request set-header X-Request-ID %[unique-id] unless { req.hdr(X-Request-ID) -m found }
    http-request set-header X-Forwarded-Proto https
    acl is_decide hdr(host) -i decide.fraud360.<BANK>.internal
    # per-source-address limit on the console/API host only; a switch on the decide host is exempt
    stick-table type ip size 200k expire 30s store http_req_rate(1s)
    http-request track-sc0 src
    http-request deny deny_status 429 if !is_decide { sc_http_req_rate(0) gt 100 }
    use_backend be_decision if is_decide
    default_backend be_gateway

backend be_gateway
    balance leastconn
    option httpchk GET /health/ready
    http-check expect status 200
    default-server inter 3s fall 3 rise 2 check
    server app-01 10.20.1.11:8080
    server app-02 10.20.1.12:8080

backend be_decision
    balance leastconn
    timeout server 1s # match the rail budget; the bank's fail policy applies beyond it
    option httpchk GET /health/ready
    http-check expect status 200
    default-server inter 2s fall 2 rise 2 check
    server dec-01 10.20.2.11:8086
    server dec-02 10.20.2.12:8086
```

Writes are never retried against a second server after the request has been sent: HAProxy retries only on connection failure by default, which is the same reason the repository's reference nginx configuration excludes non-idempotent methods from retries.

12.2 Internal VIP listeners

Dec and wrk VMs reach app services through the internal VIP. Add one listener per service port (8081–8085, 8087, 8088 to the app pool; 8086 to the decision pool):

```
listen li_ingestion
    bind 10.20.0.5:8085
```

```

balance leastconn
option httpchk GET /health/ready
http-check expect status 200
default-server inter 3s fall 3 rise 2 check
server app-01 10.20.1.11:8085
server app-02 10.20.1.12:8085
# repeat for 8081 tenant, 8082 branding, 8083 config, 8084 analytics, 8087 notification, 8088 lane-c
# and for 8086 -> dec-01/dec-02 (decision) on the internal VIP

```

This traffic is plain HTTP inside the application zone, authenticated by **INTERNAL_API_KEY**. If the bank requires encryption in transit inside the zone, terminate TLS on these listeners (**bind ... ssl crt ...**) and set the ***_SERVICE_URL** values to **https://**; the services themselves speak HTTP.

12.3 ISO 8583 pass-through (only if in scope)

```

frontend fe_iso
  mode tcp
  option tcplog
  bind 10.10.0.6:8583
  default_backend be_iso
backend be_iso
  mode tcp
  balance leastconn
  timeout server 30s
  default-server inter 3s fall 2 rise 2 check
  server dec-01 10.20.2.11:8588
  server dec-02 10.20.2.12:8588

```

12.4 keepalived

```

# /etc/keepalived/keepalived.conf (lb-01: MASTER, priority 110; lb-02: BACKUP, priority 100)
vrrp_script chk_haproxy {
  script "/usr/bin/pidof haproxy"
  interval 2
  fall 2
  rise 2
}
vrrp_instance EXT {
  state MASTER
  interface ens192
  virtual_router_id 51
  priority 110
  advert_int 1
  authentication {
    auth_type PASS
    auth_pass <vault>
  }
  virtual_ipaddress { 10.10.0.5/24 10.10.0.6/24 }
  track_script { chk_haproxy }
}
vrrp_instance INT {
  state MASTER
  interface ens224
  virtual_router_id 52
  priority 110
  advert_int 1
  authentication {
    auth_type PASS
    auth_pass <vault>
  }
  virtual_ipaddress { 10.20.0.5/24 }
  track_script { chk_haproxy }
}

```

Allow the VIPs to bind on the standby (**net.ipv4.ip_nonlocal_bind = 1**) so HAProxy starts on both nodes. Open VRRP (protocol 112) between the two LB VMs.

12.5 Exit check

```
$ curl -sS --cacert ca.pem https://fraud360.<BANK>.internal/health
$ curl -sS --cacert ca.pem -o /dev/null -w '%{http_code}\n'
https://decide.fraud360.<BANK>.internal/health/ready
$ echo 'show servers state' | socat stdio /var/lib/haproxy/stats # all UP (needs a stats socket in
'global')
# Stop haproxy on lb-01: the VIP must move and the console remain reachable within seconds.
# Stop fraud360@tenant-service on app-01: that VM must leave be_gateway; restart it: it must return.
```

13. Phases 10 and 11 — SFTP and monitoring

13.1 SFTP landing zone (sftp-01)

Each tenant has a chrooted account. The sender uploads the file, then a zero-byte marker named `<file>.done`; the intake worker ignores any file without its marker, because SFTP writes are not atomic and reading a half-delivered 400 MB file would produce a clean report over a third of the rows.

```
# groupadd -g 3601 fx-sftp
# /etc/ssh/sshd_config.d/50-fraud360-sftp.conf
Match Group fx-sftp
    ChrootDirectory /srv/fraud360/shared/drop/%u
    ForceCommand internal-sftp -u 007
    AllowTcpForwarding no
    X11Forwarding no
    PasswordAuthentication no
    AuthorizedKeysFile /etc/ssh/sftp_keys/%u

# per tenant (username = tenant id, so the chroot path matches drop/<tenant_id>/):
# useradd -g fx-sftp -G fraud360 -s /sbin/nologin -M <tenant_id>
# install -d -o root -g root -m 755 /srv/fraud360/shared/drop/<tenant_id>
# install -d -o <tenant_id> -g fraud360 -m 2770 /srv/fraud360/shared/drop/<tenant_id>/incoming
# install -d -o fraud360 -g fraud360 -m 2770 /srv/fraud360/shared/drop/<tenant_id>/archive
# install -d -o <tenant_id> -g fraud360 -m 2770 /srv/fraud360/shared/drop/<tenant_id>/failed
# install -D -m 644 -o root -g root <sender-public-key-file> /etc/ssh/sftp_keys/<tenant_id> # root-
# owned, read by sshd before the chroot
# systemctl reload sshd
```

Because the account is chrooted into its own tenant folder, a sender can reach `incoming/` only. The chroot directory itself must be root-owned and not group-writable, which is why `incoming` is a subdirectory. If the account's numeric ID is not the same on the worker VM, group access over NFS fails; keep IDs consistent as in §4.1.

Exit check: drop a small sample with its marker, wait one timer tick, and confirm it moved to `archive/` and its rows are in ingestion. A file without a marker must stay untouched and be reported.

13.2 Monitoring (mon-01)

Prometheus scrapes every service's `/metrics` (which requires the `X-Internal-Key` header), `node_exporter` on every VM, and `postgres_exporter` on the database nodes. Prometheus 2.55 or later supports per-job custom headers; on older versions put a small authenticating proxy in front.

```
# /etc/prometheus/prometheus.yml (excerpt)
scrape_configs:
  - job_name: fraud360-services
    metrics_path: /metrics
    http_headers:
      X-Internal-Key: { files: [/etc/prometheus/internal_key] } # file holds the key; mode 600
    static_configs:
      - targets:
        ['10.20.1.11:8080', '10.20.1.11:8081', '10.20.1.11:8082', '10.20.1.11:8083', '10.20.1.11:8084',
        '10.20.1.11:8085', '10.20.1.11:8087', '10.20.1.11:8088', '10.20.1.12:8080', '10.20.1.12:8081',
        '10.20.1.12:8082', '10.20.1.12:8083', '10.20.1.12:8084', '10.20.1.12:8085', '10.20.1.12:8087',
        '10.20.1.12:8088', '10.20.2.11:8086', '10.20.2.12:8086']
    - job_name: nodes
      static_configs: [{ targets: ['<every VM>:9100'] }]
    - job_name: postgres
      static_configs: [{ targets: ['10.30.0.11:9187', '10.30.0.12:9187'] }]
```

Load the alert rules from Infrastructure Requirements §11. The most important is pipeline freshness: the age of the oldest unprocessed ingestion row. Query it directly if no exporter exposes it:

```
SELECT now() - min(received_at) AS oldest_unprocessed
FROM ingestion.raw_transaction WHERE processed_at IS NULL;
```

Forward **journald** from every VM to the bank's SIEM (rsyslog or the SIEM's agent). Tracing and central log shipping are not built into the platform.

Exit check: every target is **up**; stop a worker and confirm the freshness alert fires; restore it.

14. Phase 12 — First tenant and functional verification

14.1 Administrator and tenant

- Browse to `https://<CONSOLE_HOST>`. Sign in as the seeded administrator; you are forced to change the password. Enrol MFA (TOTP).
- Remove `SEED_ADMIN_PASSWORD` from `common.env` on the app VMs and restart tenant-service on each, one at a time.
- Create the first tenant, its administrator and its users; configure SSO if in scope (OIDC discovery URL or SAML metadata). A wrong `CONSOLE_BASE_URL` shows up here as an SSO redirect to an internal address.
- Open **Roles** and review the ten starter roles the onboarding flow created. Each is the tenant's own row: edit what a role may reach or do, add roles, or delete ones the bank will not use (a role someone holds, or the last administrator role, cannot be deleted). Taking something away applies on the person's next request; giving a role more is staged and needs a different administrator to confirm, so create a second administrator first.
- Configure the tenant's email channel with the bank's SMTP relay and send a test. A tenant without a channel reports "NOT SENT – no email channel configured" rather than silently dropping the message.
- Create a machine credential for the bank's system. The token, ingestion and `/decide` contracts are in the *Fraud360 Integration Handshaking Specification* v1.3; follow it for the exact payloads.

14.2 End-to-end checks

Check	How	Pass
Token issue	Obtain a machine token from <code>/machine/token</code> via the console host	Token returned; wrong secret rejected
Ingest → detect → alert	Send a small batch of transactions through the ingestion API; wait one detection loop	Rows scored; an alert appears for a rule-matching case; the case is visible in the console
Lane A	Call <code>/decide</code> on the decide host with the token	Decision within budget; entry in the decision log; no analytics query involved
Lane C	Upload a financial-statement PDF through the console; run or wait for the Lane C worker	Statement parsed, signals scored, written to <code>lane_c</code> ; PDF present under <code>shared/lane_c_statements/<tenant>/</code> on NFS
OCR path	Upload a scanned (image) PDF	Tesseract fallback used; text extracted
File intake	SFTP drop with <code>.done</code> marker	Ingested and archived
Notifications	Trigger a case past its response window (or use a test rule)	Notification delivered through the tenant relay
Config coherence	Change a threshold through one gateway; read it through the other (pin by calling each app VM directly)	Both return the new value immediately
Secrets sealed	<code>python scripts/seal_secrets.py</code> as the owner	Reports nothing in the clear
Audit trail	Perform an administrative change; view the audit page	Event recorded with the named actor

15. Phase 13 — Acceptance tests

These produce the evidence the customer's information-security and risk teams will ask for, and replace estimates in the Infrastructure Requirements with measurements. Run them in UAT first and repeat the failover and restore tests in production before go-live.

15.1 Load and latency (BR-309 / BR-909)

`scripts/loadtest.py` measures the interval between receipt and scoring on real rows, samples queue depth throughout, and refuses to call a run sustained if the backlog did not drain. Use direct mode for detection alone, and API mode (which needs an authenticating credential) to include intake.

```
$ cd /opt/fraud360/current && export PYTHONPATH=$PWD
$ .venv/bin/python scripts/loadtest.py --tenant <uat-tenant> --mode direct --rate 200 --duration 600 --
  json /tmp/lt-200.json
$ .venv/bin/python scripts/loadtest.py --tenant <uat-tenant> --mode direct --rate 400 --duration 600 --
  json /tmp/lt-400.json # with enough detection workers
```

Criterion	Target	Source
Detection latency, receipt to alert	p99 under 5 s at the contracted peak	NFR-01
Backlog	Drains to zero after the run	Run refuses to call itself sustained otherwise
Scaling	Doubling detection workers roughly doubles sustained rate until the database becomes the limit	Unmeasured today — record the actual factor
Database	CPU, WAL volume latency and connection count within headroom at peak	Infrastructure Requirements §5.3
Lane A	Measure <code>/decide</code> p50/p95/p99 from the decide host at the expected inline rate	Rail budget 50–150 ms; first measurement of this figure

15.2 Failover and resilience

Test	Action	Expected
Load balancer	Stop HAProxy on the active LB	VIP moves; no failed console request beyond a retry
App VM	Power off app-01 under load	app-02 serves; capacity halves; no data loss
Single service	<code>systemctl stop fraud360@analytics-service</code> on app-01	That VM drained from the gateway pool; recovers on restart
Decision VM	Power off dec-01 while calling <code>/decide</code>	dec-02 answers; record the number of failed calls
Worker VM	Power off wrk-01	Backlog grows, no loss; wrk-02 drains it; enable timers on wrk-02 per §17.6
Database	<code>patronictl switchover</code> , then kill the leader	Standby promoted; VIP moves; services reconnect without restart; record the write pause
NFS	Unmount shared storage on a wrk VM	Lane C and file intake stall visibly; Lane A and detection unaffected

15.3 Backup, restore and DR (BR-908)

- **Restore to a clean VM** from the pgBackRest repository to a chosen time; start it; run `alembic current`; compare row counts and a sample of audit records with the source. Record the elapsed time.

- **Point-in-time recovery** to a time between two transactions you created deliberately; confirm the later one is absent.
- **DR promotion** at the second site: promote the standby, start the application there against it, and complete a Phase 12 check. Record the end-to-end time against the 4-hour RTO.
- Confirm that the backup repository and its encryption passphrase are retrievable by someone other than the person who set them up.

15.4 Security checks

Check	Method	Expected
Perimeter	Port scan the VIPs and every VM from a client network	Only 443 (and 8583 if used) reachable; no direct 5432, 8080–8088
TLS	<code>testssl.sh</code> or <code>nmap --script ssl-enum-ciphers</code> on the console host	TLS 1.2+ only; no weak ciphers; valid chain
Defaults gone	Sign in with <code>admin@finverge.local / ChangeMe123!</code> ; connect with <code>cp_password</code>	Both rejected
Secret scan	<code>grep</code> for default values under <code>/etc/fraud360</code> (§8.5)	Nothing
Role isolation	§7.5 tests, repeated from an app VM	Denied where expected
Metrics gate	<code>curl /metrics</code> without the key	403
MFA and lockout	Five bad sign-ins; sign in without TOTP	Account locks for 15 minutes; MFA enforced
Encryption	<code>cryptsetup status</code> on db, backup and NFS hosts; confirm Indian-region placement of every backup copy	Encrypted; localisation evidenced
Roles are enforced from data	As a tenant administrator, remove one action from a role (for example <code>reference.load</code> from Risk Manager); as that role, retry the action	Refused on the next request, with no restart; restoring it needs a second administrator to confirm
Credit and screening are gated	As Board, Chief Risk Officer and Model Risk, open Borrower Credit Health and Sanctions Screening, and call <code>GET /lane-c/<tenant>/alerts</code> and <code>/analytics/<tenant>/ofac-screen</code>	No menu entries; the API answers 403 <code>role_not_permitted</code> . An analyst can look but not upload or review
Last administrator protected	Try to remove administration from, or delete, the only administrator role	Refused (<code>last_administrator</code> or <code>role_in_use</code>)
Fresh tenant has its roles	Onboard a throwaway tenant and list its roles	Ten starter roles present, each with its grants
tenant-service outage	Stop tenant-service on both app VMs for a minute while a signed-in user keeps working	Cached roles keep serving; new tenants' first requests are refused until it returns; no request is silently allowed
Audit	Review a day's <code>platform.audit_logs</code> and SIEM forwarding	Complete and attributable

16. Phase 14 — Go-live

#	Gate	Evidence
1	All acceptance tests in §15 passed in production (or UAT with the delta stated)	Test record
2	Lane A failure policy decided and configured per rail	Signed decision
3	First backup and one restore completed; DR promotion rehearsed	Timed results
4	Alerts routed to a staffed rota; on-call can follow §17	Rota; test page acknowledged
5	Secrets in the vault with owners and rotation dates; no default remains	§8.5 output
6	Certificates have more than 90 days; renewal owner assigned	Calendar entry
7	Tenant onboarded; SSO and MFA verified; channels tested	Console screenshots / audit entries
8	Change-freeze and rollback plan for cutover agreed	Change record

Cut over integrations one at a time, starting with the lowest-risk feed (a batch SFTP drop or a non-inline CBS event stream), and only then Lane A. Lane A comes last because it sits in the payment authorisation window.

17. Day-two operations

17.1 Routine release (upgrade)

Build and verify the new release as in Phase 3. Then:

```
# 1. Safety net before any migration
$ sudo -u postgres pgbackrest --stanza=fraud360 --type=diff backup
$ psql 'host=<DB_HOST> user=postgres dbname=controlplane sslmode=require' -c "select
pg_create_restore_point('pre-$FX_VER')"
```

2. Install the new release on every VM (§9.1) WITHOUT moving the 'current' symlink yet

3. Migrate as the owner, from one worker VM

```
$ cd /opt/fraud360/releases/$FX_VER && export PYTHONPATH=$PWD DATABASE_URL=<owner url>
$ .venv/bin/python -m alembic upgrade head
$ .venv/bin/python scripts/provision_db_roles.py ...      # only if the release adds a SCHEMA (new tables
are covered by default privileges)
```

4. Roll the application, one VM at a time

```
# app-01: ln -sfn ...; systemctl restart fraud360@{tenant-service,config-service,...,gateway}; wait for
/health/ready 200
# then app-02; then dec-01, dec-02; then wrk VMs (restart detect, lanec, notify; timers pick up the new
code on next run)
```

Some migrations change data as well as structure. The role migrations rewrite `tenant.tenant_roles` for every tenant, so the backup and restore point in step 1 matter more for them than for a column addition, and the post-migration check in §7.6 should be repeated after any release that touches roles or permissions. A release that adds a gated action ships a migration granting it to the starter roles, and a tenant's own later edits are never overwritten.

Migrations are written to be applied before the new code starts and are, in practice, forward-only: rolling the schema back is a recovery action, not a normal one. Keep new columns additive and old code tolerant for one release so a code rollback does not need a schema rollback. Run `scripts/seal_secrets.py` (report mode) after a release that adds sealed columns.

17.2 Rollback

```
# Code only (schema unchanged or additive):
# ln -sfn /opt/fraud360/releases/<previous> /opt/fraud360/current
# systemctl restart fraud360@...      # rolling, as above
```

Schema damaged: restore to the 'pre-<FX_VER>' restore point into a side instance, verify, then swap (Infrastructure Requirements §10).

17.3 Rotating secrets

Secret	Effect of rotation	Procedure
Per-role DB password	None if done in order	<code>ALTER ROLE</code> → update the env file → restart that service. Brief connection errors until the restart; do one VM at a time
<code>JWT_SECRET</code>	Signs everyone out ; machine tokens invalid until re-issued	Maintenance window. Change on every VM, restart everything, tell integrators to refresh tokens
<code>INTERNAL_API_KEY</code>	Service-to-service calls fail during the mismatch window	Change on every VM and the Prometheus key file, then restart all in one window
<code>CP_ENCRYPTION_KEYS</code>	Additive. New secrets use the highest version; old ones stay readable	Prepend a new <code>2:<key></code> entry and keep <code>1:</code> for as long as any value is sealed under it. Never remove a key still in use

Secret	Effect of rotation	Procedure
Owner password	Migrations and owner jobs fail until updated	<code>ALTER ROLE cp</code> → update <code>worker-owner.env</code> and the bastion → rerun any failed job
TLS certificate	None	Replace the PEM bundle, <code>systemctl reload haproxy</code> on each LB

17.4 Scaling out

- **More detection throughput:** `systemctl enable --now fraud360-detect@3`, then 4, and so on, up to about four per worker VM; add a wrk VM beyond that. Watch database CPU and WAL latency.
- **More console capacity:** build a new app VM from Phases 1, 6; add it to `be_gateway` and the internal listeners; set `APP_REPLICAS` to the new total on every service and restart them.
- **More Lane A capacity:** same for dec VMs and `be_decision`.

17.5 Disaster recovery promotion

```
# 1. Declare the event; stop writes at the primary site if it is still partly up (stop the LBs).
# 2. At DR: promote the standby (Patroni 'failover' on the DR cluster, or pg_ctl promote for a plain standby).
# 3. Confirm the DB VIP at DR, then start tenant-service first, then the remaining services (§9.3), dec VMs, then the workers.
# 4. Re-point DNS (CONSOLE_HOST, DECIDE_HOST, DB_HOST) or move the VIPs per the bank's network plan.
# 5. Run the Phase 12 checks. Record the elapsed time against the 4-hour RTO.
# 6. Do not start workers or timers at the old primary site; rebuild it as a standby of the new primary.
```

17.6 Failing the timers over to wrk-02

```
# On wrk-01 (if reachable): systemctl disable --now $(ls /etc/systemd/system/fraud360-job-*.timer | xargs
-n1 basename) fraud360-notify
# On wrk-02:                systemctl enable --now  $(ls /etc/systemd/system/fraud360-job-*.timer | xargs
-n1 basename) fraud360-notify
```

Never run the scheduled jobs on both worker VMs at once: most are idempotent, but board packs, notifications and retention are not meant to race. The detection and Lane C workers are safe on both.

18. Troubleshooting

Symptom	Likely cause	Check / fix
Service not ready; log says <code>password authentication failed</code>	Wrong password in its env file, or <code>pg_hba.conf</code> does not allow the source	Compare env file with the vault; <code>psql</code> as that role from the same VM
<code>relation "..."</code> does not exist for a table the service should own	<code>DB_SEARCH_PATH</code> missing or wrong, or migrations not applied	Check env file against §8.3; <code>alembic current</code>
<code>permission denied for schema ...</code>	Role does not own that schema, or a job is using the wrong env file	§11 table; for a new schema re-run <code>provision_db_roles.py</code>
Service refuses to start mentioning cache or replicas	<code>APP_REPLICAS</code> above 1 with <code>CACHE_BACKEND=memory</code>	Set <code>CACHE_BACKEND=database</code>
Writing a secret raises a key error	<code>CP_ENCRYPTION_KEYS</code> unset	Set it (§8.1); never use the plaintext override
Gateway ready check fails; LB drains the VM	A local upstream is down	<code>systemctl status fraud360@<svc></code> ; fix and restart
SSO redirects to an internal address	<code>CONSOLE_BASE_URL</code> wrong	Set it to the public console origin on tenant-service
Intermittent 2-second console delays	Service URLs use <code>localhost</code> (IPv6 first)	Use <code>127.0.0.1</code> or the VIP (§8.2)
Detection backlog grows	Too few workers, a stopped worker, or a slow database	<code>systemctl status fraud360-detect@*</code> ; queue-age query (§13.2); DB CPU / WAL latency
Lane C uploads fine but never parse	Worker cannot see the PDF (NFS unmounted or ID mismatch) or Tesseract missing	<code>ls</code> the tenant folder as <code>fraud360</code> on the worker; <code>tesseract --version</code>
SFTP files never ingested	No <code>.done</code> marker, or group permissions on <code>incoming/</code>	Marker present; directory mode 2770, group <code>fraud360</code>
<code>/metrics</code> returns 403 to Prometheus	Missing or wrong <code>X-Internal-Key</code>	Check the key file and Prometheus version supports <code>http_headers</code>
Location panel empty	Geolocation provider unreachable (by design)	Infrastructure Requirements §7.4
Everyone is refused with <code>role_not_permitted</code> after a deployment	A service cannot reach tenant-service for role lookups (wrong <code>TENANT_SERVICE_URL</code> or <code>INTERNAL_API_KEY</code> , or the 8081 internal listener is missing), or the tenant has no role rows	<code>curl -H 'X-Internal-Key: <key>' http://<INT_VIP>:8081/tenants/internal/<tenant-id>/roles</code> from the failing VM; run the §7.6 query
One role is refused an action it used to do	Its permission was removed or never granted; roles are the tenant's own data	Roles page (or <code>GET /tenants/<id>/roles</code>); the audit trail shows who changed it and when
A role edit does not take effect	It raised a privilege, so it is staged until a different administrator confirms	Look for 'pending confirm' on the Roles page
Timers did not run after a reboot	Timer not enabled, or ran on the wrong VM	<code>systemctl list-timers</code> ; <code>Persistent=true</code> catches up missed runs

Appendix A — Environment variable reference

Variable	Default in code	Production value / note
DATABASE_URL	dev URL	Per service, role-specific; add <code>?sslmode=require</code> (or <code>verify-full</code>)
DB_SEARCH_PATH	all schemas	Per service as §8.3; the role-level search path matches
JWT_SECRET	dev-secret-change-me	48 random bytes; identical everywhere
JWT_EXPIRE_MINUTES	30	Short by design; revocation is on the session
INTERNAL_API_KEY	dev-internal-key	48 random bytes; identical everywhere; also gates <code>/metrics</code>
CP_ENCRYPTION_KEYS	empty	<code>version:base64(32 bytes)</code> , comma-separated; required
CACHE_BACKEND	database	Keep <code>database</code>
APP_REPLICAS	1	True total number of replicas
CACHE_GENERATION_TTL_SECONDS	0	Leave at 0
AUTO_CREATE_TABLES	true in code	<code>'false'</code> — schema belongs to Alembic
SEED_DEMO_TENANT	true in code	<code>'false'</code>
SEED_ADMIN_EMAIL / PASSWORD	published defaults	Named admin; one-time password; remove afterwards
SEED_ADMIN_MUST_CHANGE_PASSWORD	true	Keep <code>true</code>
CONSOLE_BASE_URL	<code>http://127.0.0.1:8080</code>	Public console origin (HTTPS)
*_SERVICE_URL	localhost / 127.0.0.1	Per §8.2
RATE_LIMIT_PER_SECOND / RATE_LIMIT_BURST	20 / 40	Per gateway process; tune with the load test
LANE_C_STORAGE_DIR	<code>./data/lane_c_statements</code>	Shared storage path
INGEST_DROP_ROOT	drop	Shared storage path
REPORT_SPOOL_DIR	spool	Shared storage path
GEOLOCATE_URL	<code>http://ip-api.com/json</code>	Internal service, or leave unreachable
NARRATIVE_POLISH_ENABLED, LANE_C_LLM_ENABLED	false	Leave off unless the customer opts in and <code>llm-01</code> exists
NARRATIVE_LLM_BASE_URL, LANE_C_LLM_BASE_URL, LLM_BASE_URL	<code>http://localhost:11434/v1</code>	Point at <code>llm-01</code> only if used
ISO8583_*	empty / None	Client id/secret, tenant, TZ offset, <code>ISO8583_FAIL_OPEN</code> (no default)
CP_ALLOW_PLAINTEXT_SECRETS	unset	Never set

Appendix B — Port and unit map

Unit	Port	Hosts	Enable
fraud360@gateway	8080	app	enable
fraud360@tenant-service	8081	app	enable (start first)
fraud360@branding-service	8082	app	enable
fraud360@config-service	8083	app	enable
fraud360@analytics-service	8084	app	enable
fraud360@ingestion-service	8085	app	enable
fraud360@decision-service	8086	dec	enable

Unit	Port	Hosts	Enable
fraud360@notification-service	8087	app	enable
fraud360@lane-c-service	8088	app	enable
fraud360@iso8583-gateway	8588 (TCP), 8589 (HTTP)	dec	only if in scope
fraud360-detect@1..N	—	wrk	enable
fraud360-lanec	—	wrk	enable
fraud360-notify	—	wrk-01	enable (one host only)
fraud360-job-*.timer	—	wrk-01	enable (one host only)